

# Algorithms for Lattice Fermions User Workshop 2022

14-18th Feb 2022

Pre-conference event: Installation  
11th Feb 2022



# **ALF**

## Installation notes

---

ALF Team

February 2022

University of Würzburg

# Table of contents

## 1. Introduction

- The ALF project

- Prerequisites

## 2. ALF Installation

- Installing ALF prerequisites

- Installing ALF

- Testing ALF

## 3. pyALF installation

- Installing pyALF prerequisites

- Downloading and testing pyALF

## Introduction

---

# Introduction - The ALF project

The ALF project contains the following public parts:

- **ALF**: our main package, responsible for performing auxiliary-field quantum Monte Carlo simulations; also its **documentation** can be found here
- **pyALF**: provides a Python interface and scripts, as well as Jupyter notebooks
- **ALF\_Tutorial**: stores ALF's tutorial, including solutions to the proposed exercises
- **ALF\_Presentations**: where, for instance, these notes are made available
- **ALF\_code**: the main package's legacy version, ALF 1.0

Version-control is done using Git and they are available in the GitLab repository:

<https://git.physik.uni-wuerzburg.de/ALF>.

These notes cover the installation for ALF and pyALF.

# Introduction - Prerequisites

To run ALF and pyALF you need the following installed in your machine:

For ALF:

- Make
- A Fortran compiler,  
such as `gfortran` or `ifort`
- Blas library
- Lapack library
- Python3
- MPI libraries (optional)
- For HDF5
  - C++ preprocessor (default: `g++`)
  - Curl
  - gzip development libraries

For pyALF

- The following Python3 libraries
  - `h5py`
  - `numpy`
  - `pandas`
  - `matplotlib`
  - `numba`
  - `scipy`
  - `tkinter`
  - `ipywidgets`
  - `ipyml`
  - `f90nml`
- JupyterLab or Jupyter Notebook

## ALF Installation

---

## Installing ALF prerequisites – Linux

To install the required libraries and compiler in one of the following Linux distributions, enter in your terminal:

- Debian/Ubuntu/Linux Mint:

```
sudo apt install make gfortran libblas-dev liblapack-dev \  
python3 libopenmpi-dev g++ curl libghc-zlib-dev git
```

- Red Hat/Fedora/CentOS:

```
sudo yum install make gfortran libblas-devel liblapack-devel \  
python3 libopenmpi-dev g++ curl zlib git
```

- OpenSuSE/SLES:

```
sudo zypper install make gcc-fortran lapack-devel \  
python3 libopenmpi-devel gcc-c++ curl zlib-devel git
```

- Arch Linux:

```
pacman -S make gcc-fortran lapack python3 curl zlib git
```



### MacOS

For MacOS, Xcode and the Apple developer tools should be installed.

gfortran for MacOS can be found at:

<https://gcc.gnu.org/wiki/GFortranBinaries#MacOS>,

where detailed information on how to install the package can be found.

### Windows 10/11

For Windows 10/11, we suggest using the Windows subsystem for Linux to install Ubuntu and the follow the instructions for Ubuntu. For installation instructions see:

<https://docs.microsoft.com/en-us/windows/wsl/install>

### Older Windows versions

For older Windows versions, you can use Cygwin, found at <https://www.cygwin.com/>, which provides a Unix-like environment for Windows.

# Installing ALF

With its dependencies in place, we can proceed to the proper ALF installation.

ALF's source code can be downloaded by entering the following in a shell:

```
git clone https://git.physik.uni-wuerzburg.de/ALF/ALF.git
```

To set the necessary environment variables and compiler directives run the script `configure.sh`:

```
source configure.sh [MACHINE] [MODE] [STAB] [Devel|HDF5]
```

where `MACHINE` can be, e.g., `GNU`, for choosing `gfortran` (see ALF's documentation).

When activating `HDF5` the first time, it will be downloaded and compiled automatically, which takes several minutes.

In order to compile the libraries, the analysis routines and the QMC program at once, just execute the single command:

```
make
```

When the compilation is over you are in position to run simulations with ALF.

Now let us use ALF:

1. Create a data directory with the content of `Start` found in the folder `Scripts_and_Parameters_files`:

```
cp -r $ALF_DIR/Scripts_and_Parameters_files/Start ./Run && cd ./Run/
```

2. Run an executable Hamiltonian, e.g.:

```
$ALF_DIR/ALF.out
```

3. Perform default analysis<sup>1</sup>:

```
$ALF_DIR/Analysis/ana.out *
```

4. Check results. For instance, `Ener_scalJ` should be around  $-30$  for default parameters.

---

<sup>1</sup>The `analysis.sh` bash script from earlier versions of ALF, run without arguments, is still available in the `Start` directory.

## pyALF installation

---

## Installing pyALF prerequisites

We recommend installing the necessary Python packages via pip:

```
sudo apt install python3-pip python3-tk
```

```
pip install h5py numpy pandas matplotlib numba scipy \  
jupyterlab ipywidgets ipympl f90nml
```

## Downloading and testing pyALF

With pyALF's dependencies taken care of, now only the package itself is missing.

The package can be downloaded by entering the following in a shell:

```
git clone https://git.physik.uni-wuerzburg.de/ALF/pyALF.git
```

Finally, append pyALF's path to the environment variable PYTHONPATH, add the folder with command line scripts to PATH and defining ALF\_DIR by, e.g., adding to .bashrc:

```
export PYTHONPATH="/local/path/to/pyALF:$PYTHONPATH"
export PATH="/local/path/to/pyALF/py_alf/cli:$PATH"
export ALF_DIR="/local/path/to/ALF"
```

So that Python can find the py\_alf module, pyALF can find ALF and the command line tool can be conveniently executed from the shell.

With this, one should be able to run

```
minimal_ALF_run.py
```

from the Unix shell to test whether everything has been set up correctly.